

CENTRO UNIVERSITÁRIO DE MACEIÓ - UNIMA/AFYA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Isabella Monte Santa Cruz

Ivanildo Marques de Souza Filho

Ramon Freire Alencar

**Gerenciador de Tabela de Símbolos: apresentação de
relatório técnico**

Maceió
2026

Isabella Monte Santa Cruz

Ivanildo Marques de Souza Filho

Ramon Freire Alencar

**Gerenciador de Tabela de Símbolos: apresentação de
relatório técnico**

Relatório técnico apresentado como requisito
parcial para obtenção de aprovação na
disciplina Compiladores no Curso de Ciência
da Computação, do Afya/Maceió.

Prof. Amaury Nogueira Neto

Maceió

2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. METODOLOGIA DE IMPLEMENTAÇÃO.....	4
2.1 ESTRUTURA GERAL.....	4
2.2 CONTROLADOR DE ESCOPOS.....	5
2.3 DECLARAÇÃO DE VARIÁVEIS.....	5
2.4 BUSCA DE VARIÁVEIS.....	6
3. CASOS DE TESTE.....	6
3.1 Caso de Teste 1 – Declaração de Variável no Escopo Global.....	6
3.2 Caso de Teste 2 – Busca de Variável Existente.....	7
3.3 Caso de Teste 3 – Criação de Novo Escopo.....	7
3.4 Caso de Teste 4 – Busca em Escopos Aninhados.....	7
3.5 Caso de Teste 5 – Remoção de Escopo.....	7
3.6 Caso de Teste 6 – Busca Após Remoção de Escopo.....	8
4. DESENVOLVIMENTO.....	8

1. INTRODUÇÃO

A Tabela de Símbolos é uma das estruturas fundamentais utilizadas em compiladores e interpretadores. Sua principal função é armazenar informações sobre identificadores presentes em um programa, como variáveis, funções, constantes e seus respectivos atributos, incluindo tipos, escopo e localização na memória.

Durante o processo de compilação, a tabela de símbolos é consultada constantemente para verificar se variáveis foram declaradas corretamente, se estão sendo utilizadas dentro de seus escopos válidos e para recuperar informações necessárias às etapas de análise semântica e geração de código.

Este projeto consiste na implementação de um Gerenciador de Tabela de Símbolos utilizando Pilhas e Hash Tables, permitindo o gerenciamento de escopos aninhados. O sistema oferece operações para declaração de variáveis, busca de identificadores e controle de entrada e saída de escopos, simulando o comportamento encontrado em compiladores reais.

O desenvolvimento foi realizado na linguagem Python devido à simplicidade oferecida para manipulação de estruturas de dados, especialmente listas e dicionários.

2. METODOLOGIA DE IMPLEMENTAÇÃO

A implementação foi baseada na utilização de duas estruturas de dados principais: pilhas e tabelas hash.

2.1 ESTRUTURA GERAL

O sistema foi desenvolvido por meio da classe denominada `TabelaDeSimbolos`, responsável por gerenciar todos os escopos existentes durante a execução.

A estrutura principal consiste em uma pilha de escopos, onde cada escopo é representado por um dicionário Python.

Exemplo conceitual:

```
[  
  {'x': 'int', 'nome': 'string'},  
  {'y': 'float'}  
]
```

Nesse exemplo:

O primeiro dicionário representa o escopo global.

O segundo dicionário representa um escopo interno.

Cada chave corresponde ao nome de uma variável e cada valor corresponde ao seu tipo.

2.2 CONTROLADOR DE ESCOPOS

O gerenciamento dos escopos é realizado por meio de uma pilha. Quando um novo bloco de código é iniciado, um novo escopo é criado através do método:

entrar_escopo()

Esse método adiciona um novo dicionário vazio ao topo da pilha. Quando o bloco é encerrado, o método:

sair_escopo()

remove o escopo mais interno, eliminando todas as variáveis declaradas naquele contexto. Essa abordagem reproduz o comportamento dos compiladores durante o processamento de estruturas como funções, laços e blocos condicionais.

2.3 DECLARAÇÃO DE VARIÁVEIS

A declaração de variáveis é realizada pelo método:

declarar(variavel, tipo)

Ao receber o nome da variável e seu tipo, o método verifica se a variável já existe no

escopo atual. Caso não exista, a variável é inserida no dicionário correspondente ao escopo ativo. Como por exemplo:

declarar("idade", "int")

Resultado:

idade → int

Essa verificação impede a duplicação de declarações dentro de um mesmo escopo.

2.4 BUSCA DE VARIÁVEIS

A recuperação de informações é realizada através do método:

buscar(variavel)

A busca ocorre do escopo mais interno para o mais externo. Dessa forma, caso existam variáveis com o mesmo nome em diferentes níveis de escopo, o sistema sempre encontrará primeiro a declaração mais próxima. Esse comportamento corresponde ao mecanismo de escopo léxico utilizado na maioria das linguagens de programação modernas.

3. CASOS DE TESTE

Para validar o funcionamento do sistema, foram executados diferentes cenários de teste.

3.1 Caso de Teste 1 – Declaração de Variável no Escopo Global

Entrada:

tabela.declarar("x", "int")

Resultado Esperado:

Variável 'x' declarada como 'int'.

Resultado Obtido:

A variável foi inserida corretamente no escopo global.

3.2 Caso de Teste 2 – Busca de Variável Existente

Entrada:

```
tabela.buscar("x")
```

Resultado Esperado:

'x' encontrada com tipo 'int'.

Resultado Obtido:

A busca retornou corretamente o tipo associado à variável

3.3 Caso de Teste 3 – Criação de Novo Escopo

Entrada:

```
tabela.entrar_escopo()
```

Resultado Esperado:

>>> Novo escopo criado

Resultado Obtido:

Um novo escopo foi adicionado ao topo da pilha.

3.4 Caso de Teste 4 – Busca em Escopos Aninhados

Entrada:

```
tabela.declarar("y", "float")
```

```
tabela.buscar("x")
```

Resultado Esperado:

'x' encontrada com tipo 'int'.

Resultado Obtido:

O sistema localizou corretamente a variável no escopo externo.

3.5 Caso de Teste 5 – Remoção de Escopo

Entrada:

tabela.sair_escopo()

Resultado Esperado:

>>> Escopo removido.

Resultado Obtido:

O escopo interno foi removido da pilha.

3.6 Caso de Teste 6 – Busca Após Remoção de Escopo

Entrada

tabela.buscar("y")

Resultado Esperado

Erro: 'y' não encontrada

Resultado Obtido:

O sistema informou corretamente que a variável não estava mais disponível após a remoção do escopo.

4. DESENVOLVIMENTO

O projeto atingiu os objetivos propostos ao implementar um Gerenciador de Tabela de Símbolos utilizando uma pilha de Hash Tables para representar escopos aninhados.

A solução permitiu simular funcionalidades presentes em compiladores reais, incluindo declaração de variáveis, busca hierárquica de identificadores e gerenciamento de escopos. Os testes realizados demonstraram o correto funcionamento das operações implementadas.

Além de reforçar conceitos fundamentais de compiladores, o desenvolvimento também proporcionou a aplicação prática de estruturas de dados como pilhas e tabelas hash, amplamente utilizadas em sistemas de software e linguagens de

programação.

Como possível evolução do projeto, poderiam ser adicionadas funcionalidades como armazenamento de funções, verificação de tipos e suporte a múltiplos atributos associados aos identificadores.